# *Programación Lógica*

*Clase 9 . Introducción al paradigma lógico*

*PARADIGMAS DE PROGRAMACION* *UTN - La Plata*

## Temario

1. Características del Paradigma
2. Base de conocimiento
3. Consultas
4. Motor de inferencias
5. Tipos de reglas
6. Usos y aplicación
7. Prolog
8. Actividades

## 1. Características del paradigma

- En el paradigma lógico se trabaja en forma descriptiva: el programa NO indica cómo resolver el problema sino establece relaciones entre las entidades del mismo, las cuales describen ***qué*** hacer. Por esta razón se lo encuadra en la programación declarativa.

- Un programa lógico NO tiene un algoritmo que indique los pasos a seguir para llegar al resultado, sino que está formado por **expresiones lógicas que declaran o describen la solución** y es el sistema interno quien proporciona la secuencia de control en que se utilizan esas expresiones.

## 1. Características del paradigma

Tiene como característica principal el uso de reglas lógicas o predicados de 1º orden para inferir o derivar conclusiones a partir de datos.

Conociendo los datos y las condiciones del problema, la ejecución de un programa consiste en demostrar un objetivo a partir de las relaciones declaradas.

En este paradigma, un algoritmo lógico se construye:

Especificando **una base de conocimiento**

sobre la que se realizan **consultas**

y aplicando un **mecanismo de inferencia o deducción** sobre dicha base, **se derivan o infieren** conclusiones.

## 2. Base de conocimiento

- Se construye con **predicados de 1° orden llamados Cláusulas de Horn**.

- Cada cláusula permite deducir conclusiones a partir del conjunto de datos del problema.

- Una cláusula de Horn tiene la siguiente forma:

c :=p1, p2, p3, p4,…….pk

donde **c** es una conclusión o consecuente y las $\mathbf{p_i}$ son las premisas o antecedentes

Se dice que **c** es verdadera si son verdaderas cada una de las premisas $\mathbf{p_i}$.

(Tiene el formato si p entonces q)

## 2. Base de conocimiento

- Está formada por hechos y reglas.

- Los hechos representan enunciados o predicados siempre verdaderos que muestran relaciones entre los datos del problema. (axiomas)

- Las reglas muestran relaciones entre los datos, los hechos y otras reglas, y permiten deducir o demostrar objetivos.

- Hechos → hombre(juan).
  hombre(tomás).
  mujer(ana).
  progenitor(ana, juan).

- Reglas → mortal(X):- humano(X).

  cabeza cuerpo

  que significa: X es mortal si X es humano

  Si X es humano entonces X es mortal.

  madre(X,Y):- mujer(X), progenitor(X,Y).

## 3. Consultas

- Una consulta es un objetivo a ser demostrado por comparación o pattern matching contra la base de conocimiento.

- Se trata de aparear la consulta con algún hecho o con la cabeza de una regla.

- La consulta puede ser de dos tipos:

    - validación (cuya respuesta es Si o No)

    - una búsqueda (que retorna los valores que hacen verdadero al objetivo).

## 3. Consultas

- Si puede demostrar la consulta u objetivo, responde:
  - **Yes** (si es una validación)
  - o **el/los valor/es** que satisface/n la consulta (en caso de búsqueda).

- Si no puede demostrar el objetivo, responde **No**, que tiene dos significados:
  - que es falso
  - o que no puede deducirlo con la información contenida en la base de conocimiento

## 3. Consultas

- Ejemplo:

?- hombre(juan)

Yes

?- hombre(luis)

No

?hombre(X)

X=juan;

X=tomás;

No

BC

hombre(juan).

hombre(tomás).

mujer(ana).

mujer(pilar).

progenitor(ana, juan).

Observación:
Si colocamos un ; después de un resultado, sigue mostrando otros resultados posibles hasta agotarlos todos.
Si colocamos un . el proceso de búsqueda se corta.

## 3. Consultas

- Si la consulta posee variables, las mismas se ***asocian o ligan*** con los valores de los hechos o con los argumentos de las cabezas de las reglas. Esto se denomina ***unificación.***
- No existe el concepto de asignación de valor a las variables. Tampoco el concepto de variable como celda de memoria, sino como variables matemáticas. Una vez que fueron ligadas a un valor, dicho valor no cambia. No hay efectos laterales.(propiedad de transparencia referencial)
- Al aparearse las variables de una consulta con la cabeza de una regla, las variables se ligan y luego se aplica la regla: se reemplaza la cabeza de la regla por su definición y debe demostrarse cada sub-objetivo de la misma. Este proceso de demostración se denomina ***resolución***.

## 3. Consultas

- Ejemplo:

?-hombre(X)
X=juan;
X=tomás;
No
?- madre(ana,Z)
Z=juan;
Z=maría;
No

BC

hombre(juan).

hombre(tomás).

mujer(ana).

mujer(pilar).

progenitor(ana, juan).

progenitor(ana,maría).

madre(X,Y):- mujer(X), progenitor(X,Y).

Se aparea la consulta con la cabeza de la regla:
liga X→ ana y Z→ Y, reemplaza en el cuerpo de la regla y la aplica.

Madre(ana, Z):- mujer(ana),progenitor(ana,Z)
ahora tiene 2 subobjetivos nuevos a demostrar.

## 3. Consultas

Se pueden hacer 3 tipos de consultas:

- Sin variables
  - ?gusta(maria, juan)  a maria le gusta juan  Validación

- Con variables
  - ?gusta(maria, X)  quién le gusta a maría  Búsqueda

- Con variable anónima: para saber si existe algún objeto que haga verdadera la consulta
  - ?gusta(maria,_)  hay alguien que le gusta a maría. La respuesta es Si o No

## 3. Consultas: reversibilidad el predicado

- Se pueden introducir valores en la consulta esperando que el motor de inferencias encuentre las variables de salida que apareen o unifiquen con la consulta.

```
?- progenitor(ana, X)
X=juan;
No
```

De quién es progenitor ana?

BC
mujer(ana).
hombre(juan).
progenitor(ana, juan).

- Recíprocamente también puedo introducir un valor de salida y esperar que el motor encuentre qué valores de entrada son adecuados para satisfacer esa salida.

```
?- progenitor(Y, juan)
Y=ana;
No
```

quién es el progenitor de juan?

No todos los predicados son reversibles!!!

## 4. Motor de inferencias

- Es quien controla la ejecución del programa lógico a partir de una consulta.

- Recorre la base de datos desde el principio y permite ***derivar o deducir*** un objetivo en base a dos mecanismos: ***recursión y backtracking***.

- Gracias al backtracking, si falla 1 camino de demostración, retrocede a un punto previo (deshace la última sustitución) y reinicia el proceso de demostración. Cuando agotó todos los caminos posibles, termina la demostración.

## 4. Motor de inferencias

eshombre(juan).

eshombre(pedro).

eshombre(luis).

quiere(juan,maria).

quiere(juan, elena).

quiere(pedro, raquel).

quiere(pedro, belen).

?-eshombre(X),quiere(X,Y)

X=juan Y=maría;
X=juan Y=elena;
X=pedro  Y=raquel;
X=pedro  Y=belen;
No

Proceso de resolución: evalúa de izq a derecha.

Toma 1er objetivo: eshombre(X):

Liga X=juan  y reemplaza en el segundo objetivo

?quiere(juan,Y)  busca todos los posibles valores de Y

X=juan Y=maría;

X=juan Y=elena;  No hay más alternativas posibles

Por backtracking retrocede al objetivo previo: eshombre(X)

Liga X=pedro y reemplaza en el segundo objetivo

?quiere(pedro,Y)

X=pedro  Y=raquel;

X=pedro  Y=belen;  No hay más alternativas posibles

Por backtracking retrocede al objetivo previo: eshombre(X)

Liga X=Luis y reemplaza en el segundo objetivo

?quiere(Luis,Y)

No encuentra ninguna cláusula en la BC que satisfaga el segundo objetivo, termina el proceso y muestra No.

## 5. Tipos de reglas

- Simples
  - notrabaja(belen).
  - bueno(pedro).
  - cuida(belen, pedro):- notrabaja(belen), bueno(pedro). → regla sin variables

  } hechos

- Con variables
  - humano(juan).
  - humano(ana).
  - mortal(X):- humano(X). → regla

  } hechos

- Recursivas
  - antepasado(X,Y):- progenitor(X,Y).
  - antepasado(X,Y):- antepasado(X,Z), progenitor(Z,Y).

  Observar que el caso base va siempre primero.

## Cómo programar en este paradigma – Usos y aplicaciones

- A) Modelar el problema mediante una Base de Conocimiento formada por hechos que muestren las relaciones de los datos de dicho problema.
- B) Formular luego reglas que se cumplan en el mundo del problema modelado.
- C) Formular consultas al sistema para que demuestre o deduzca la respuesta a partir de la base de conocimiento usando resolución.

- Se aplica a:
  - Inteligencia Artificial
  - Sistemas expertos
  - Robótica
  - Procesamiento de Lenguaje Natural
  - Compiladores
  - Bases de datos deductivas
  - Acceso a bases de datos desde páginas web

# 7. PROLOG (PROgrammation in LOGique)

- Lenguaje interpretado, que consta de un compilador y un intérprete, un Shell (utilidad para probar programas) y una biblioteca
- Sintaxis:
  - Constantes y predicados comienzan con minúscula
  - Variables comienzan en mayúscula
  - Hechos y predicados terminan en punto
  - Se usa ; en una consulta para ver más resultados
  - El <enter> o el . finaliza la consulta
  - Los comentarios se empiezan con %

## 7. PROLOG (PROgrammation in LOGique)

- Predicados predefinidos:
  - true
  - fail
  - var(X) devuelve V si X es una variable sin instanciar
  - nonvar(X) devuelve T si X tiene valor asignado
  - integer(X) devuelve V si X es una variable entera
  - float(X) devuelve V si X es una variable real
  - number(X) devuelve V si X es una variable numérica
  - write(X) imprime el valor de X
  - read(X) ingresa un valor a X
  - == compara si son iguales
  - \== compara si son distintos

  Operadores: + - / * X mod Y

# 7. PROLOG (PROgrammation in LOGique)

Ejemplos de reglas:

El operador **is** sirve para asignar valor o para comparar, dependiendo su uso.

- suma(X,Y,Z):- number(X), number(Y), Z is X+Y.

- par(X):- integer(X), 0 is X mod 2.
  Esta regla devuelve true si 0 es el resultado de hacer X mod 2
  par(X,Res):- integer(X), 0 is x mod 2, Res is true.
  par(X,Res):- not (integer(X)), Res is false.
  par(X,Res):- integer(X), 1 is X mod 2, Res is false.

- impar(X):- integer(X),1 is X mod 2.

- igual(X,Y):- X==Y.

- cuad(X,Y):- nonvar(X), number(X), Y is X*X.

## 7. PROLOG (PROgrammation in LOGique)

Ejemplos de reglas:

```
mayor(X,Y,Z):- X>Y, Z is X.
mayor(X,Y,Z):- X<Y, Z is Y.
mayor(X,Y,Z):- Z is 0.
```

Otra forma de hacerlo:

```
mayor(X,Y,X):- X>Y.
mayor(X,Y,Y):- X<Y.
mayor(X,Y,0):- X==Y.
```

```
posit(X):- X>0.
posit(X):-X<0,false.
posit(X):-X==0,false.
```

```
posit(X,Z):- X>0, Z is true.  %se usa la hipótesis de mundo cerrado y no se escribe la rama falsa
```

# 7. PROLOG (PROgrammation in LOGique)

Ejemplo de hechos y reglas:

%hechos

- gusta(martin,negro).
- gusta(diana,verde).
- gusta(lucas,azul).
- gusta(cecilia,blanco).
- gusta(soledad,violeta).
- color(pantalón,negro).
- color(pantalón,azul).
- color(camisa,verde).
- color(bufanda,violeta).
- color(saco,azul).

%reglas

% X usa la prenda Y si le gusta el color de dicha prenda

- usa(X,Y) :- gusta(X,Z), color(Y,Z).

Consultas:

?-usa(lucas,R)

?- usa(martin,camisa)

?- usa(X,bufanda)

Cómo se llama la propiedad aplicada en la última consulta?

## 1. Listas

- Son conjuntos de elementos del mismo o de distinto tipo.
- Las listas se denotan con corchetes, separando los elementos por comas. Por ej. [1,2,3,4]
- Están divididas en cabeza (X) y cuerpo (Xs) → [X | Xs]
- La lista vacía se denota [ ].
- Se recorren recursivamente.
- Contienen términos (variables, constantes o estructuras) en general. Es posible anidar listas.

Ejemplos: [1,2,3,4] , [a,b,c,d]

- [1,'a', X,[1,2,3]]
- [[1,2],[3,4],[5,6] ]

## 2. Listas: ejemplos

**Longitud de una lista**

```
long([ ],C):-C is 0.
long([X|Xs], C):- long(Xs,C1), C is C1 + 1.
```

**Suma de los elementos de una lista**

```
sum([ ],0).
sum([X|Xs], S):-sum(Xs,S1), S is S1 + X.
```

**Cantidad de elementos pares de una lista**

```
par(X):- integer(X), 0 is X mod 2.
impar(X):-integer(X), 1 is X mod 2.
cantP([ ],0).
cantP([X|Xs], S):- par(X), cantP(Xs,S1), S is S1 + 1.    L=[1,2,3]
cantP([X|Xs], S):- impar(X), cantP(Xs,S1), S is S1.
```

## 2. Listas: ejemplos

**Verifica si todos los elementos de una lista son hombres**

```
hombre(juan).
hombre(pedro).
hombres([]).
hombres([X|Xs]):-hombre(X),hombres(Xs).
```

**Verifica si todos los elementos de una lista son iguales**

```
todosIg([]).
todosIg([X]).
todosIg([X,Y|Xs]):- X==Y, todosIg([Y|Xs]).
todosIg([X,Y|Xs]):- X \==Y, false.
```

## 2. Listas: ejemplos

**Cuenta los elementos de una lista que son hombres**

```
hombre(juan).
hombre(pedro).
hombres([],0).
hombres([X|Xs], Cont):-hombre(X),hombres(Xs, C), Cont is C +1.
hombres([X|Xs], Cont):- not (hombre(X)) ,hombres(Xs, C), Cont is C.
```

**Cantidad de elementos mayores a 5 de una lista**

```
cantMay5([ ],0).
cantMay5([X|Xs], S):- X > 5, cantMay5(Xs,S1), S is S1 + 1.
cantMay5([X|Xs], S):- X < 5, cantMay5(Xs,S1), S is S1 .
cantMay5([X|Xs], S):- X == 5, cantMay5(Xs,S1), S is S1 .
```

## 2. Listas: ejemplos

**Imprimir los elementos de una lista**

```
mostrar([ ]).
mostrar([X|Xs]):- write(X), nl, mostrar(Xs).
```

**No existe un elemento en una lista**

```
noesta(X, [ ]).
noesta(X, [Y|Ys]):- X \==Y, noesta(X,Ys).
noesta(X, [Y|Ys]):- X ==Y, false.
```

## 2. Listas:ejemplos.

**Existe un elemento en una lista**

```
member(X,[ ]):-fail.
member(X,[ Y|Ys]):- X==Y, true.
member(X,[Y|Ys]) :- X\==Y, member(X,Ys).
```

**Añadir un elemento a una lista (al pcpio)**

```
add(X,[ ],[X]).
add(X,L,[X|L]).
```

Otra forma

```
add(X,[ ],[X]).
add(X,[Y|Ys],[X,Y|Ys]).
```

## Verificar si hay algún elemento positivo en la lista

```
unPosit([ ]):- false.
unPosit([X|Xs]):- X>0.
unPosit([X|Xs]):- X < 0, unPosit(Xs).
unPosit([X|Xs]):- X=0, unPosit(Xs).
```

**O bien:**

```
unPosit([ ]):- false.
unPosit([X|Xs]):- X>0.
unPosit([X|Xs]):- (X < 0; X=0), unPosit(Xs).
```

Se usa el punto y coma como OR

## 2. Listas: ejemplos

**Borrar un solo elemento de una lista**

```
del(X,[ ],[ ]).
del(X,[Y|Ys],Ys):- X==Y.
del(X,[Y|Ys],[Z|Zs]):- X\==Y, del(X,Ys,Zs), Z is Y.
```

## 2. Listas: ejemplos

**Imprimir invertidos los elementos de una lista**

```
mostrar([ ]).
mostrar([X|Xs]):- mostrar(Xs), write(X),nl.
```

```
?mostrar([3,6,2])
```

con el llamado recursivo primero recorre la lista hasta llegar a vaciarla, y por backtracking imprime de atrás hacia adelante

```
2
6
3
```

## 2. Listas: ejemplos

**Dada una lista de apellidos de alumnos, y conociendo sus promedios, imprimir los nombres de aquellos alumnos cuyo promedio es mayor a 7.**

```
prom(mia,5).
prom(juan,8).
prom(ana,6).
prom(lucas,9).

impriProm([]).
impriProm([X|Xs]):-prom(X,Val),Val>7, write(X),nl, impriProm(Xs).
impriProm([X|Xs]):-prom(X,Val),Val<7,impriProm(Xs).
impriProm([X|Xs]):-prom(X,Val),Val=7,impriProm(Xs).
```

## 2. Listas: ejemplos

**Definir un programa lógico que dada una lista de alumnos:**

- **retorne la lista de aquellos que promocionan. Se sabe que un alumno promociona si cumplió con la asistencia solicitada y su nota es mayor o igual a 7.**
- **Imprima la lista de aprobados. Para aprobar la nota obtenida debe ser mayor o igual que 6.**

**La base de conocimiento contiene hechos sobre las notas de cada alumno y su asistencia.**

nota(juan,6).

nota(pedro,8).

nota(luis,9).

nota(ana,7).

nota(andrea,6).

nota(tomas,4).

nota(belen,2).

asistencia(juan).

asistencia(pedro).

asistencia(ana).

asistencia(andrea).

asistencia(belen).

## 2. Listas: ejemplos

```
promociona(X):-nota(X,Z),(Z>7;Z=7),asistencia(X).  %uso el ; como OR %
% promociona(X):-nota(X,Z),Z<7,fail.    Opcional, falla por la nota
% promociona(X):-fail.                  Opcional, falla por asistencia
```

## 2. Listas: ejemplos

```
promociona(X):-nota(X,Z),(Z>7;Z=7),asistencia(X).  %uso el ; como OR %
% promociona(X):-nota(X,Z),Z<7,fail.    Opcional, falla por la nota
% promociona(X):-fail.                  Opcional, falla por asistencia

listaPromo([],[]).
listaPromo([X|Xs],[X|Ys] ):- promociona(X), listaPromo(Xs,Ys).
listaPromo([X|Xs],L):- not(promociona(X)),listaPromo(Xs,L).
```

## 2. Listas: ejemplos

```
promociona(X):-nota(X,Z),(Z>7;Z=7),asistencia(X).  %uso el ; como OR %
% promociona(X):-nota(X,Z),Z<7,fail.    Opcional, falla por la nota
% promociona(X):-fail.                  Opcional, falla por asistencia

listaPromo([],[]).
listaPromo([X|Xs],[X|Ys] ):- promociona(X), listaPromo(Xs,Ys).
listaPromo([X|Xs],L):- not(promociona(X)),listaPromo(Xs,L).

listaAprob([]).
listaAprob([X|Xs]):-nota(X,Z), (Z>6;Z=6), write(X),nl, listaAprob(Xs).
listaAprob([X|Xs]):-nota(X,Z), Z<6, listaAprob(Xs).
```

